

Linux Text Editors & Regex - Study Notes

I. Regular Expressions (Regex)

Regular expressions are powerful pattern-matching tools used across programming languages for text processing.

Basic Regex Patterns

Test String Used:

```
sally sells seashells  
by the seashore
```

I.1 Beginning of Line (^)

- **Pattern:** `^by`
- **Matches:** Lines starting with "by"
- **Example:** Matches "by the seashore" (second line)

I.2 End of Line (\$)

- **Pattern:** `seashore$`
- **Matches:** Lines ending with "seashore"
- **Example:** Matches "by the seashore" (ends with seashore)

I.3 Any Single Character (.)

- **Pattern:** `b.`
- **Matches:** "b" followed by any single character
- **Example:** Matches "by" (b + y)

I.4 Bracket Notation ([])

Character Sets:

- **Pattern:** `d[iou]g`
- **Matches:** "d" + any character from [i,o,u] + "g"
- **Examples:** dig, dog, dug

Negation with ^:

- **Pattern:** `d[^i]g`
- **Matches:** "d" + any character EXCEPT "i" + "g"
- **Examples:** dog, dug (but NOT dig)

Character Ranges:

- **Pattern:** `d[a-c]g`
- **Matches:** "d" + any character from a to c + "g"
- **Examples:** dag, dbg, dcg

Case Sensitivity:

- **Pattern:** `d[A-C]g`
 - **Matches:** "d" + uppercase A to C + "g"
 - **Examples:** dAg, dBg, dCg (but NOT dag, dbg, dcg)
-

2. Text Editors Overview

Two dominant text editors in Linux:

- **Vim:** Lightweight, quick, universally available
- **Emacs:** Powerful, extensible, feature-rich

Both are command-line based and installed by default on most Linux distributions.

3. Vim (Vi Improved)

3.1 Starting Vim

```
vim                # Start vim
vim filename.txt    # Open specific file
```

3.2 Vim Modes

- **Command Mode:** Default mode for navigation and commands
- **Insert Mode:** For typing text

- **Visual Mode:** For selecting text

3.3 Search Patterns

Forward Search (/)

```
/pretty
```

- **Function:** Search forward for "pretty"
- **Navigation:** Press `n` for next match, `N` for previous match
- **Example:** In "My pretty file is very pretty" - finds both instances

Backward Search (?)

```
?pretty
```

- **Function:** Search backward for "pretty"
- **Navigation:** Same as forward search
- **Example:** In "My pretty file is very pretty" - finds last "pretty" first

3.4 Navigation Commands

Basic Movement

```
h or ←    # Move left one character
j or ↓    # Move down one line
k or ↑    # Move up one line
l or →    # Move right one character
```

Example Navigation:

```
Hello World
↑k (up)
←h (left) Hello World →l (right)
↓j (down)
```

3.5 Entering Insert Mode

```
i          # Insert before cursor
a          # Append after cursor
```

```
o      # Insert on new line below
O      # Insert on new line above
A      # Append at end of line
```

Example:

```
Original: Hello World
Press 'i' at 'W': Hello |World (cursor before W)
Press 'a' at 'o': Hello Wo|rld (cursor after o)
Press 'o' at end: Hello World
                    | (new line below)
```

Exit Insert Mode: Press `Esc`

3.6 Editing Commands

Text Manipulation

```
x      # Delete character under cursor
dd      # Delete entire current line
y      # Copy (yank) selected text
yy      # Copy entire current line
p      # Paste before cursor
```

Example Editing Session:

```
Original text: The quick brown fox
Place cursor on 'q' and press 'x': The uick brown fox
Press 'dd' on any line: (entire line deleted)
Press 'yy' then 'p': (line copied and pasted)
```

3.7 Saving and Exiting

```
:w      # Save file
:q      # Quit vim
:wq     # Save and quit
:q!     # Quit without saving
ZZ      # Save and quit (shortcut)
```

Undo/Redo

```
u          # Undo last action
Ctrl-r     # Redo last action
```

Example Save Session:

```
1. Edit file
2. Press Esc (enter command mode)
3. Type :w (save)
4. Type :q (quit)
Or simply: :wq (save and quit)
```

4. Emacs

4.1 Starting Emacs

```
emacs          # Start emacs
emacs filename.txt  # Open specific file
```

4.2 Emacs Notation

- **C-[letter]**: Ctrl + letter (e.g., C-x = Ctrl+x)
- **M-[letter]**: Meta/Alt + letter (e.g., M-x = Alt+x)

4.3 File Operations

Saving Files

```
C-x C-s      # Save current file
C-x C-w      # Save as (specify new filename)
C-x s        # Save all open files
```

Opening Files

```
C-x C-f      # Open file (or create new if doesn't exist)
```

Example File Operations:

```
C-x C-f → Enter filename → myfile.txt  
(Opens myfile.txt or creates it if it doesn't exist)  
C-x C-s → Saves the file
```

4.4 Buffer Management

Buffer Navigation

```
C-x b          # Switch to another buffer  
C-x →          # Cycle right through buffers  
C-x ←          # Cycle left through buffers  
C-x k          # Close current buffer
```

Window Management

```
C-x 2          # Split window horizontally  
C-x 1          # Return to single window  
C-x o          # Switch between split windows
```

Example Buffer Workflow:

1. Open file1.txt: C-x C-f file1.txt
2. Open file2.txt: C-x C-f file2.txt
3. Switch between them: C-x b file1.txt
4. Split view: C-x 2 (see both files)
5. Move between splits: C-x o

4.5 Text Navigation

Advanced Movement

```
C-↑           # Move up one paragraph  
C-↓           # Move down one paragraph  
C-←           # Move left one word  
C-→           # Move right one word  
M->           # Move to end of buffer
```

Example Text Navigation:

Original: "Hello world. This is a test paragraph.

Another paragraph here."

C-↓ from first line moves cursor to "Another paragraph"

C-→ from "Hello" moves cursor to "world"

4.6 Cut and Paste Operations

Selection and Editing

C-space	# Start text selection
C-w	# Cut (kill) selected text
C-y	# Paste (yank) text

Example Cut/Paste:

1. Position cursor at start of word "Hello"
2. Press C-space (start selection)
3. Move cursor to end of word (Hello is selected)
4. Press C-w (cut "Hello")
5. Move cursor elsewhere
6. Press C-y (paste "Hello")

4.7 Exiting and Help

C-x C-c	# Exit emacs
C-h C-h	# Help menu
C-x u	# Undo last action

Example Exit Process:

1. Press C-x C-c
2. If unsaved changes exist, emacs prompts: "Save file? (y/n)"
3. Type y to save, n to discard changes
4. Emacs closes

Quick Reference Summary

Vim Quick Commands

```
Start: vim filename
Insert: i, a, o, O, A
Navigate: h,j,k,l
Edit: x, dd, yy, p
Save/Exit: :w, :q, :wq, ZZ
Search: /pattern, ?pattern
```

Emacs Quick Commands

```
Start: emacs filename
Files: C-x C-f, C-x C-s, C-x C-w
Buffers: C-x b, C-x k, C-x 2
Edit: C-space, C-w, C-y
Exit: C-x C-c
Help: C-h C-h
```

Regex Quick Patterns

<code>^pattern</code>	# Start of line
<code>pattern\$</code>	# End of line
<code>.</code>	# Any character
<code>[abc]</code>	# Character set
<code>[^abc]</code>	# Not in character set
<code>[a-z]</code>	# Character range